Concerning the `annotations` field, you want to provide rich context for the human operator receiving a notification based on the alert. Consider that the notification may reach the person on call at an early hour, and the last thing you want in this situation is to start digging for references, wasting valuable time to stop the bleeding, while still trying to wake up. For example, you could, in addition to the `summary` and `description` fields we defined in the preceding example, add the following:

- A link to a dashboard that shows the time series for the relevant time frame
- An assessment of who or what is impacted (whether these are real customers or canaries)
- A link to a runbook, for known issues and contributing factors. If you can follow these links from your phone without having to log in to, say, eight systems to get them to render, you've done a good job.

Now that we know what alerting rules are and how to define them, let's see how to set up Prometheus to include the alerting rules and establish the connection to an Alertmanager. This is done as follows in the Prometheus configuration file prometheus.yml:

```
global:
  scrape_interval: 30s
  external_labels:
    environment: dev
    region: us-west-1
rule_files:
  - alert.rules
alerting:
  alertmanagers:
  - static_configs:
    - targets: ["alertmanager.local:9093"]
scrape_configs:
  ...
```

Adds external labels to provide context for the Prometheus instance

Points to the file where the alerting rules are defined

Configures the Alertmanager Prometheus should use

ALERTMANAGER

Let's now move on to the Alertmanager (http://mng.bz/yQ7p). First, we have a look at how the Alertmanager processes alerts. This is done in the so-called notification pipeline, consisting of the following phases:

1. *Inhibition*—Suppress notifications for alerts if other (related) alerts are already firing. For example, think of the preceding case, where you don't want pod-level notifications when you're already notified of a cluster data plane issue.
2. *Silencing*—Suppress notifications ad hoc, in a proactive manner. For example, if you're upgrading nodes in an Kubernetes cluster, you already know in advance that nodes will go down temporarily, so you don't want to get a notification for this.
3. *Routing*—Decide who gets a notification.
4. *Grouping and throttling*—Ensure you get one notification per event.